College of Engineering and Information Technology

Flying Radar: A Drone-Based Software-Defined Radar

Commercial In Confidence

ENGI 4900: Final Report

Authors: Hoi Yan Leung (a1890865)
Lysander Tonkin (a1888005)
Kyle Miller (a1887019)
Axel Stevens (a1887987)

Supervisor: Brian Ng

Advisors: Luke Rosenberg
Chow Yii Pui

Group: 2026s1-EME.EE-BWN-UG-14487

13 Nov 2026

36/36 Pages

Summary of AI Use

Categories of artificial intelligence (AI) tool use in this assignment are as indicated in the following table.

AI tool use category	**Used**
Spelling	✓
Grammar	✓
Other copy editing	
Literature review support	
Visuals	
Substantive editing	
Idea generation	
Translation	
Research technical support	✓
No AI tool was used	

Acknowledgements

The authors acknowledge and sincerely thank Dr Brian Ng for his supervision, technical guidance, and ongoing support throughout the project. The authors also thank Luke Rosenberg and Chow Yii Pui for their advice and technical input during the development of the drone-mounted radar platform.

The authors acknowledge Lockheed Martin Australia Advanced Systems & Technologies for supporting the project, providing the radar payload, and contributing technical context for the development of a replicable UAV-mounted software-defined radar platform.

The authors also thank the Adelaide University academic and technical staff, ENGI 4900 course coordinators, and moderation team for their guidance, feedback, and support throughout the project.

Contents

List of Figures

List of Tables

List of Acronyms

Acronym	Definition
3D	three dimensional
AC	alternating current
AI	artificial intelligence
API	application programming interface
ASA	acrylonitrile styrene acrylate
AST	Advanced Systems & Technologies
CLI	command-line interface
CPU	central processing unit
C-UAS	counter-uncrewed aerial system
CAD	computer-aided design
COTS	commercial off-the-shelf
CW	continuous wave
DC	direct current
DEM	digital elevation model
FMCW	frequency modulated continuous wave
GPIO	general purpose input/output
GPR	ground penetrating radar
GPS	global positioning system
HALE	high-altitude long-endurance
IED	improvised explosive device
InSAR	interferometric synthetic aperture radar
IQ	in-phase and quadrature
JST	Japan solderless terminal

LiPo	lithium-polymer
PCB	printed circuit board
PLA	polylactic acid
RF	radio frequency
SAR	synthetic aperture radar
SBC	single-board computer
SDR	software-defined radar
SLD	single-line diagram
UAV	uncrewed aerial vehicle
USB	universal serial bus

Contribution Statement

This contribution statement records the specific contribution of each team member to the technical work and preparation of this progress report. Contributions are grouped according to the simplified CRediT categories of Approach, Technical output, Writing, and Project management.

Table CS1: Declaration of group contributions to the report.

Author	Team Role	Approach	Technical Output	Writing	Project Management

Table CS2: Acknowledgement of AI Tool Use

Group member name	**Specific sub-sections or paragraphs where AI was used**	**AI Tool Use Category/ies**
Hoi Yan Leung	Software implementation documentation and selected grammar/spelling checks.	Spelling; grammar; research technical support
Lysander Tonkin	Selected spelling and grammar checks in drafted report text.	Spelling; grammar
Kyle Miller	Selected spelling and grammar checks in drafted report text.	Spelling; grammar
Axel Stevens	Selected spelling and grammar checks in drafted report text.	Spelling; grammar

Table CS3: Group Member Comments

Group member name	**Comments**
Hoi Yan Leung	Oversaw technical SDR work, developed theory section of the report and made substantial technical progress towards SDR and SBC interfacing.
Lysander Tonkin	Coordinated the initial group formation and refined the technical objectives for task delegation. Made progress towards mechanical system design.
Kyle Miller	Coordinated the development of the report. Completed editing and refinement. Made progress towards the power system for the project.
Axel Stevens	Reworked literature review subsections and completion plan. Made progress towards data synchronisation.

By signing below students are acknowledging that the contributions recorded above are a true representation of the contribution made by each individual.

Table CS4: Signature Acknowledgement of Contribution

Name	**Student ID**	**Signature**
Hoi Yan Leung	a1890865	
Lysander Tonkin	a1888005	
Kyle Miller	a1887019	
Axel Stevens	a1887987	

4 Methods, Results and Discussion

4.1 Objective 4: Synchronise Logging of Flight Telemetry and Radar Measurements

4.1.1 Introduction

To synchronise flight telemetry and radar measurements, a pilot-initiated control signal must be interpreted, processed, and transmitted through the system. The signal travels from the pilot transmitter to the receiver, then to the flight controller, and finally to the SBC, which triggers SDR logging. This signal path is shown in Figure **??**.

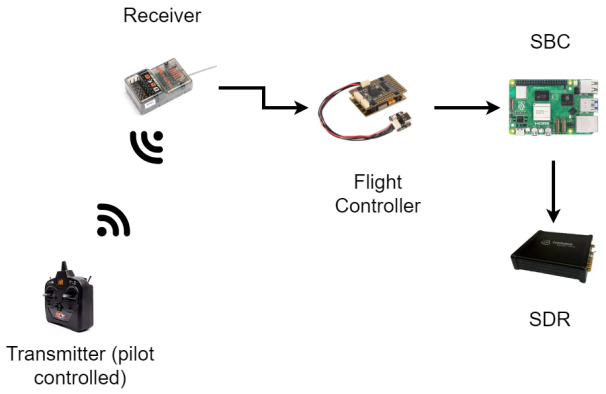
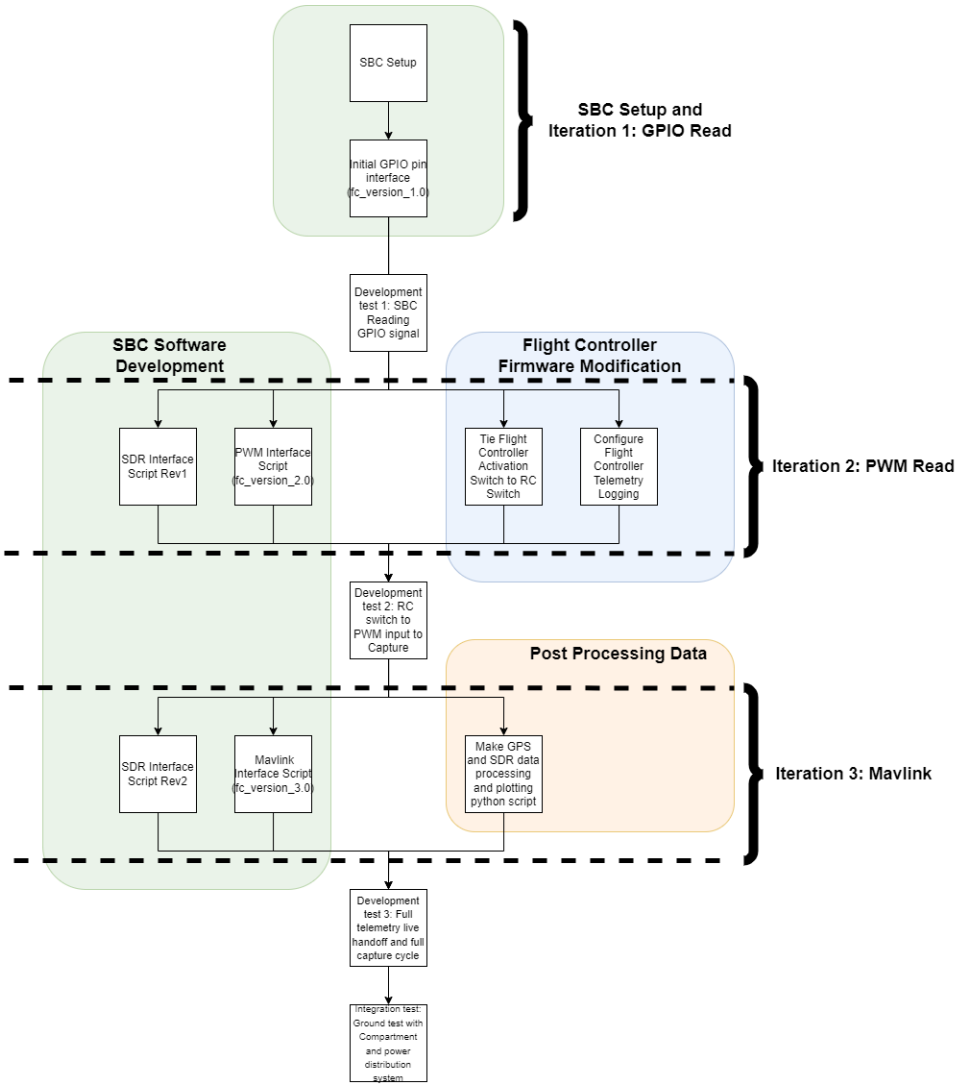


Figure 4.1: Activation signal flow starting at the pilot transmitter and ending at the SDR.

Progress toward this objective has included researching and procuring a suitable SBC, completing its initial setup, and conducting a communication trial between the SBC and a simulated flight-controller signal.

4.1.2 Methodology



SBC Setup and Iteration 1: GPIO Read

SBC Setup

First, requirements of the SBC were identified. The hardware requirements for the SBC were derived from the SDR datasheet, which specifies the need for a high-speed USB 2.0 interface (480 Mb/s) [**?**]. Additionally, the SBC should provide sufficient processing capability to support potential future expansion of onboard processing capability, placing requirements on CPU performance and RAM capacity.

Consequently, a Raspberry Pi 5 with 16 GB of RAM was selected and procured. In addition to satisfying the hardware requirements, Raspberry Pi devices were selected because of their well-supported software ecosystem and prior use in drone-mounted radar research applications [**?**, **?**, **?**, **?**]. A protective case and active cooler were also acquired to secure the device on board the drone and manage central processing unit (CPU) temperature during SDR operation.

The Raspberry Pi 5 was physically assembled and configured with a lightweight Ubuntu 22.04 operating system. The installation process used an imager to write the Ubuntu disk image to a 64 GB microSD card. The card was then inserted into the Raspberry Pi 5, and the system was booted to verify the operating-system installation.

A remote development workflow was then configured to develop and test the Python script used to read the general purpose input/output (GPIO) pin. Native development on the Raspberry Pi was avoided due to its lower performance compared with a desktop computer. Instead, secure shell (SSH) was configured and integrated with an integrated development environment (IDE) to enable remote development.

SSH provides a secure remote connection between a host and a remote computer. In this setup, the desktop computer was used as the development machine, and the Raspberry Pi 5 was used as the remote host. The setup commands are shown in Listing **??**. These commands install the SSH server, enable the SSH service at boot, and allow incoming SSH connections through the firewall. The successful SSH connection is shown in Appendix X.

Listing 4.1: The commands used to setup SSH connection between the Raspberry Pi and a remote computer.

```
sudo apt install openssh-server
sudo systemctl enable ssh
sudo ufw allow ssh
```

The SSH connection was then integrated with VS Code using the Microsoft *Remote - SSH* extension. The SSH configuration file on the development laptop was updated with the Raspberry Pi's IP address and username, after which VS Code was connected to the Raspberry Pi through the remote SSH interface.

Iteration 1: GPIO Read

To allow the Raspberry Pi to read GPIO pins, the required GPIO packages were installed. The installation command is shown in Listing **??**. The package `python3-lgpio` provides Python bindings and low-level GPIO access, and `gpiod` provides terminal tools for inspecting and manipulating GPIO pins.

Listing 4.2: Bash command used to install python GPIO library.

```
sudo apt install python3-lgpio gpiod
```

The `lgpio` package is used interface with GPIO pins on a level too low in abstraction for the application. The package to use for interfacing with the pins at a higher abstraction level was called `rpi-lgpio`, which uses `lgpio` as a backened. The package can only be installed with `pip`, so a Python virtual environment was created. The virtual environment setup and installation of `rpi-lgpio` is shown in Listing **??**

Listing 4.3: commands used to create the python virtual environment and install `rpi-lgpio`

```
wget http://abyz.me.uk/lg/lg.zip
unzip lg.zip
cd lg
make
sudo make install
python3 -m venv gpio-env
source gpio-env/bin/activate
pip install rpi-lgpio
```

After installing the packages, a Python script was created to read a GPIO pin and respond when an external component pulls the signal high. The script is shown in Listing **??**.

Listing 4.4: Python script used to run on the Raspberry Pi to monitor one of the GPIO pins.

```
#!/usr/bin/env python3
import RPi.GPIO as GPIO # Uses rpi-lgpio backend automatically
from time import sleep
import signal

# Setup
GPIO_PIN = 17
GPIO.setmode(GPIO.BCM)
GPIO.setup(GPIO_PIN, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)

print("Sensing␣3.3V␣High...")

try:
    while True:
        if GPIO.input(GPIO_PIN):
            print("Pin␣High")
            sleep(1)
except KeyboardInterrupt:
    print("Stopping...")
finally:
    GPIO.cleanup()
pause()
```

To verify the program, an Arduino Uno R3 was used to generate an input signal for a GPIO pin. Because the Raspberry Pi 5 uses 3.3 V logic and the Arduino Uno R3 outputs 5 V logic, a resistor divider was used to reduce the signal to approximately 3.3 V and protect the Raspberry Pi.

This process concluded the development of the first software that was made for the flight controller.

Iteration 2: PWM Read

After the first software iteration was tested and confirmed working as expected, a new iteration was created. This iteration now reads in the specific signal sent from the flight controller output to the GPIO pins, which is pulse-width modulation (PWM). This iteration was split into several parallel tasks. On the flight controller side, outputting a specific PWM channel from an activation switch on the transmitter was a task, as was configuring the flight controller for telemetry logging. On the SBC side, a refined script to interface with the flight controllers signals was made, and setting up the SBC to interface with the Radar (including the function to run the SDR capture and setting up the software requirements to run the SDR) was it's own task. These four tasks are outlined in this section.

Tie Flight Controller Activation Switch to RC Switch

Before doing anything, the flight controller had to be conencted to a Desktop PC to be able to interface with the device properly. This was done by installing and using Mission Planner.

Upon configuring, the task then became to tie a PWM output channel to an RC switch on the transmitter. THis PWM output channel would then be read by the SBC, which would activate/deactivate the radar capture accordingly.

On the transmitter in possession, there were 8 physical switches. The switch that was deemed

suitable was tied to RC channel 15 using the transmitter settings.

In Mission Planner, RC Channel 15 was then tied to servo output 8. After configuring, the output signal was scoped with an oscilloscope and confirmed working as intended.

This development concluded with a successful tying of the transmitter physical switch to a servo output channel on the flight controller, enabling the transmitter to have a physical switch dedicated to radar capture. What to bring up:

1. Signal flow point that we are at
2. What PWM is
3. Tying the PWM channel on the drone and outputting from the flight controller

Configure Flight Controller Telemetry Logging

It was also imperative to gain an understanding of how the flight controller recorded telemetry and configure the logging in such a way to suit the application. That was the objective of this section.

It was found that when the host PC connects to the flihgt controller, the flight controller makes a .tlog file and continues to write to the .tlog file until the connection is terminated.

The telemetry log only records global positioning system (GPS) when the drone is armed. This setting did not suit the project, as the drone needed to be disarmed for the majority of the project. To enable GPS logging while disarmed, a parameter was changed in the config file in Mission Planner.

1. How the flight controller logs telemetry
2. Setting when to record telemetry
3. getting the telemetry logs off from the flight controller

PWM Interface Script

At this stage of development, the flight controller interfaces with the SBC via PWM. Whether a signal is logic HIGH or logic LOW is determined by the pulse width of the transmission. For the flight controller signal, a pulse width below 1300 μs is logic LOW, and a pulse width above 1700 μs is logic HIGH. This is the logic embedded into the script that the SBC uses to interface with the flight controller.

The input logic is controlled by the `RCInput` class, which is shown in Listing **??**

Listing 4.5: RCInput class used to control PWM input logic

```
1  import RPi.GPIO as GPIO
2  import time
3
4  from .config import RC7_PIN, RC8_PIN
5  from .config import PWM_HIGH, PWM_LOW
6
7
8  class RCInput:
9
10     def __init__(self):
11
12         self.rc7_pulse_width = 0
13         self.rc8_pulse_width = 0
14
15         self.rc7_high = False
```

```
        self.rc8_high = False

        self.rc7_rising_time = None
        self.rc8_rising_time = None

        self.rc7_last_low = 0
        self.rc8_last_low = 0

        GPIO.setup(RC7_PIN, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)

        GPIO.setup(RC8_PIN, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)

        GPIO.add_event_detect(RC7_PIN, GPIO.BOTH, callback=self.rc7_callback)

        GPIO.add_event_detect(RC8_PIN, GPIO.BOTH, callback=self.rc8_callback)

    def rc7_callback(self, channel):

        if GPIO.input(channel):

            # Rising edge
            self.rc7_rising_time = time.monotonic_ns()

        else:

            # Falling edge
            if self.rc7_rising_time is not None:

                falling_time = time.monotonic_ns()

                self.rc7_pulse_width = (falling_time - self.rc7_rising_time) / 1000

                self.rc7_rising_time = None

                if self.rc7_pulse_width > PWM_HIGH:

                    self.rc7_high = True

                elif self.rc7_pulse_width < PWM_LOW:

                    self.rc7_high = False

    def rc8_callback(self, channel):

        if GPIO.input(channel):

            # Rising edge
            self.rc8_rising_time = time.monotonic_ns()

        else:

            # Falling edge
            if self.rc8_rising_time is not None:
```

```
69
70                falling_time = time.monotonic_ns()
71
72                self.rc8_pulse_width = (falling_time - self.rc8_rising_time) / 1000
73
74                self.rc8_rising_time = None
75
76                if self.rc8_pulse_width > PWM_HIGH:
77
78                    self.rc8_high = True
79
80                elif self.rc8_pulse_width < PWM_LOW:
81
82                    self.rc8_high = False
```

The main flow of the program is controlled by the main.py script, which is shown in Listing **??**. The script initialises the Raspberry Pi GPIO interface and creates instances of the RC input, radar, GPS, and display classes. It then continuously monitors the RC channels, starting a radar capture when the appropriate input transitions high and activating GPS when the corresponding input is high. Runtime and capture information are written to output.txt, while the display is periodically updated with the current system state. When the program is interrupted or terminated, it performs a shutdown routine that cleans up the environment.

Listing 4.6: Main file of PWM interface script

```
1  #!/usr/bin/env python3
2
3  import RPi.GPIO as GPIO
4  import time
5  import signal
6
7  from files.rc_input import RCInput
8  from files.radar import Radar
9  from files.gps import GPS
10 from files.display import display
11 from files.config import DISPLAY_REFRESH
12
13 # ============================================================
14 # GLOBAL STATE
15 # ============================================================
16
17 start_time = time.monotonic()
18
19 # ============================================================
20 # INITIALISE
21 # ============================================================
22
23 GPIO.setmode(GPIO.BCM)
24
25 rc = RCInput()
26
27 radar = Radar()
28
```

```
gps = GPS()

# ============================================================
# OUTPUT
# ============================================================


def update_output(gps, radar):
    with open("output.txt", "w") as file:
        file.write(f"GPS␣Activation␣time:␣{gps.activation_time}\n")
        file.write(f"Radar␣capture␣times:␣{radar.capture_times}\n")


# ============================================================
# MAIN LOOP
# ============================================================


def main():

    global drone_armed
    global start_time

    print("Starting␣drone␣radar␣interface...")
    open("output.txt", "w") # overwrite previous file

    try:

        while True:

            runtime = time.monotonic() - start_time

            # ------------------------------------------------
            # Radar Capture
            # ------------------------------------------------

            if rc.rc7_high and rc.rc7_last_low:

                radar.start_capture(runtime)
                update_output(gps, radar)
                radar.stop_capture()
                rc.rc7_last_low = 0

            if not rc.rc7_high:
                rc.rc7_last_low = 1

            # ------------------------------------------------
            # GPS
            # ------------------------------------------------

            if rc.rc8_high:

                gps.activate()
```

```
                update_output(gps, radar)

            # ------------------------------------------------
            # DISPLAY
            # ------------------------------------------------

            display(rc, radar, gps, runtime)

            time.sleep(DISPLAY_REFRESH)

    except KeyboardInterrupt:

        shutdown()


# ============================================================
# SHUTDOWN
# ============================================================


def shutdown():

    print("\nCleaning up...")

    GPIO.cleanup()


def signal_handler(signal_number, frame):

    shutdown()

    exit(0)


signal.signal(signal.SIGINT, signal_handler)

signal.signal(signal.SIGTERM, signal_handler)


# ============================================================
# START
# ============================================================

if __name__ == "__main__":

    main()
```

1. The PWM interface
2. Overview of the classes
3. Main function flow

SDR Interface function Rev1

The PWM interface script was modified to enable the use of Bash commands to activate the

script that initiates radar capture. Following this, the class to handle the radar capture script was implemented. This function is shown in Listing **??**. The Radar class manages the state of radar captures and records basic capture information. When a capture is started, it prevents duplicate captures, increments the capture count, records the capture time, and adds it to a list of previous capture times. The `stop_capture()` method ends the active capture, while the commented `subprocess.run()` shows where the external radar acquisition command can be executed.

Listing 4.7: Radar capture class implemented in pwm interface script

```
import time
import subprocess


class Radar:

    def __init__(self):

        self.capturing = False
        self.capture_count = 0
        self.last_capture_time = None

        self.capture_times = []

    def start_capture(self, runtime):

        if self.capturing:
            return

        self.capturing = True

        self.capture_count += 1

        self.last_capture_time = runtime

        self.capture_times.append(self.last_capture_time)

        print("Radar capture started")

        # Start radar here
        # subprocess.run(
        # """
        # sudo ./build-arm64/pupradar_capture --firmware firmware/SDR_USB_FW.hex --
            duration 5 --fc-low 24.00e9 --fc-high 26e9 --sweep-time 1 --samp-num 4 --
            out ~/SDR-data/0209_pi/0209_2GHz_128_0-5ms_loopback_rx1_01 --rx 1
        # """,
        # shell=True,
        # executable="/bin/bash",
        # check=True,
        # )

    def stop_capture(self):

```

```
        if not self.capturing:
            return

        self.capturing = False

        print("Radar capture stopped")
```

1. Getting Python to run shell code
2. Setting the sticky capture function
3. The execution capture cycle

Iteration 3: Mavlink

Likewise from the second iteration, a single python 3 program on the SBC handled both the flight controller and the SDR in one loop.

Again, the code was split by device, a `Drone` class, which handled the flight controller interface, a `Radar` class, which handled the radar interface, and a small `main.py` script to run the overall program.

The benefit to the program being one one system was the fact that all the timing was able to use a single clock, which was `runtime` (seconds elapsed since the start of the program). Both the telemetry log and the radar used this value, and shared the same time base which is what made synchronisation possible.

Mavlink Interface

The SBC interfaced with the drone via Mavlink, a communication protocol used to interface between flight controllers and desktop computers (which the Raspberry Pi 5 running Ubuntu has a similar architecture to). The physical connection was over universal serial bus (USB) serial, and the software interface was handled by the `pymavlink` python package.

The script blocked until it received a `HEARTBEAT` signal which confirmed the link and gave the target system and component IDs to be used by later commands. Instead of relying on default stream rates, the script sent `MAV_CMD_SET_MESSAGE_INTERVAL` commands to request each message at a fixed rate. The message ID's of the key messages is shown in Table **??**.

Table 4.1: MAVLink messages requested from the flight controller and their use.

Message	ID	Rate	Used for
GPS_RAW_INT	24	1 Hz	Fix type, satellites, latitude, longitude
GLOBAL_POSITION_INT	33	5 Hz	Altitude (AMSL and relative), groundspeed
ATTITUDE	30	5 Hz	Roll, pitch, yaw
BATTERY_STATUS	147	1 Hz	Voltage, current, remaining capacity
VFR_HUD	74	2 Hz	Requested but not parsed
RC_CHANNELS	65	5 Hz	Channel 15 (radar trigger)
HEARTBEAT	0	Default	Armed state, flight mode

on each loop in the `main.py` file, update() drained every queued message with non blocking `recv_match(blocking=False)`, and so the program was never stalled by a slow linkm as the stored value was always the latest received.

Raw Mavlink fields were converted to International System of Units (SI) units upon reception, including converting lattitude and longitude from degE7 to degrees, altitude from milimetres to metres, and voltage from millivolts to volts.

The armed status of the drone was read from the `MAV_MODE_FLAG_SAFETY_ARMED` bit of `HEARTBEAT.base_mode`, and the flight mode was decoded with `mode_string_v10`. The `gps_ready` state was changed based on whether a fix to three or more satellites was present.

Same from iteration 2, the radar was triggered from a switch on the pilot's transmitter which was mapped to channel 15. However, instead of reading in a PWM signal output from the flight controller, the signal was read through the `RC_CHANNELS` MAVlink message.

The trigger logic remained the same; rising edge detection. A capture was requested by the main script to the radar only when the pulse width was above 1800 . The capture logic was sticky; the switch had to be seen as low before a new capture could be requested (which was a pulse width below 1200 μs). The sticky logic prevented one switch from firing repeatedly, ignored the noise in the invalid range, and prevented a switch left high at startup from requesting a capture. Both pulse width parameters can be changed easily in the config file. Buffering was not used; if the radar was still busy, the request was rejected and a message printed, instead of the request being queued. The reason for this was

SDR Interface

GPS and SDR Processing and Plotting

4.1.3 Results and Discussion

Development test 1: GPIO Read

Development test 2: RC Switch to PWM Capture Cycle

Development test 3: Full telemetry live handoff and full capture cycle

Ground Test: Full Integration with other subsystems on the ground demonstrating full capture cycle and logging

4.1.4 Summary

Statement of AI Use

Spelling and Grammar

The project team used ChatGPT (https://chat.openai.com/) and Claude (https://claude.ai/) for this Progress Report to check spelling and grammar in selected text drafted by the team. The use of AI tool was limited to identifying spelling and grammar errors. The tools were not used to draft new report sections, produce summaries or paraphrases, restructure the report, interpret results, draw conclusions. The project team reviewed all suggestions before making any changes and retained copies of the original text submitted to the AI tools.

The following prompt was used in May 2026:

> Please review the following paragraph and identify spelling and grammar errors only. Do not rewrite the text, change the meaning, improve the flow, shorten sentences, add content, or make style or substantive editing suggestions.

Initials of group members who used this support: HL, LT, KM, AS.

Research Technical Support

The project team used Claude Code to analyse the legacy MATLAB driver, implement radar commands, and produce code documentation. All code and documentation generated by AI underwent human review and approval. AI use in this form was approved verbally by Dr Brian Ng. The following prompt was used in April 2026:

> Document all functions in the following file using Doxygen-style comments:
>
> `C:\Users\eliza\SDR_main\SDR_main\linux\src\UsbDevice.cpp`
>
> Documentation requirements:
>
> - Add a Doxygen comment block above every function definition.
> - Include:
>   - `@brief`
>   - `@param` for all parameters
>   - `@return` where applicable
>   - `@note` or `@warning` if relevant
> - Keep descriptions concise but technically clear.

Initials of group members who used this support: HL